5. Transform (posición, rotación y escala)

El componente **Transform** es uno de los elementos esenciales en Unity, ya que define la **posición**, **rotación** y **escala** de cualquier GameObject en la escena. Todos los objetos, tanto visibles como lógicos, poseen un Transform.

5.1. Posición

La posición se representa mediante un **Vector3**, incluso en proyectos 2D (donde Z suele permanecer en 0).
Acceder y modificar la posición es una de las operaciones más comunes en los scripts.

Ejemplo tomado del script **Jugador**:

```
transform.position += velocidad * (Vector3)direccion *
Time.deltaTime;
```

Aquí se usa una suma de vectores para desplazar al jugador cada frame.

Otros ejemplos comunes:

```
transform.position = new Vector3(0, 5, 0);

transform.position = transform.position + Vector3.right * 2f;
```

5.2. Rotación

La rotación también se representa en **Vector3**, como ángulos de Euler (grados) sobre los ejes X, Y y Z.

Unity proporciona dos formas principales de establecer rotación:

a) Mediante eulerAngles

```
transform.eulerAngles = new Vector3(0, 0, 90);
```

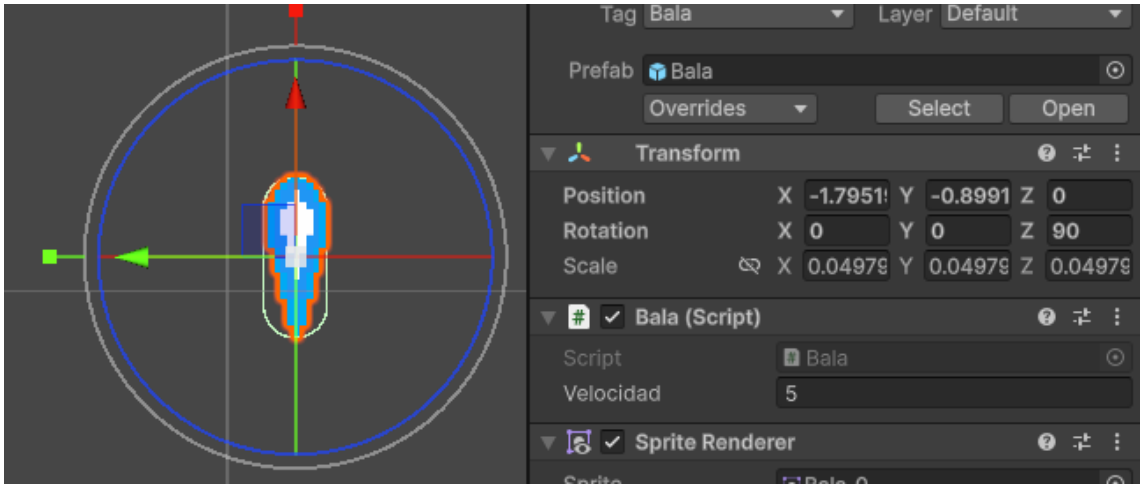
b) Mediante Quaternion

Los Quaternions evitan problemas de interpolación, pero para la mayoría de proyectos 2D se puede trabajar con eulerAngles o funciones simplificadas.

Ejemplo tomado del script **Jugador** cuando dispara en distintas direcciones:

```
Instantiate(prefabBala, transform.position + new Vector3(0, 0.5f, 0), Quaternion.Euler(0, 0, 90));
```

Aquí se ajusta la rotación del proyectil según la dirección del disparo. En los objetos 2D, el eje de rotación frecuentemente es el Z, aunque en ocasiones se rota en Y para reflejar 180º a un objeto.



5.3. Escala

La escala determina el tamaño del GameObject y se expresa en Vector3.

```
transform.localScale = new Vector3(2, 2, 1);
```

En 2D se suele modificar solo X e Y:

- localScale.x controla el ancho
- localScale.y controla la altura
- localScale.x = -1 suele usarse para reflejar un objeto

Ejemplo real del script **EnemigoPajaro**, donde se invierte la escala del pájaro según el movimiento:

```
transform.localScale = new Vector3(-1, 1, 1);
```

Esto permite girar el sprite sin alterar la rotación del Transform.

5.4. Transform como jerarquía

El Transform también define las relaciones **padre–hijo** entre objetos.

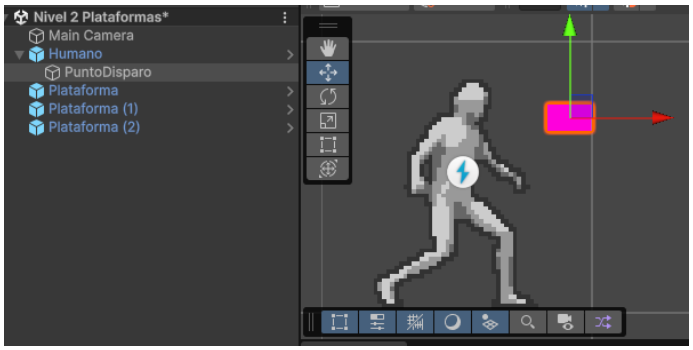
```
childObject.transform.parent = this.transform;
```

Beneficios:

- Que un objeto siga a otro automáticamente

- Agrupar elementos visuales
- Construir interfaces, armas, enemigos compuestos, etc.

Ejemplo real del proyecto:
Los objetos del HUD están agrupados bajo un Canvas, que actúa como padre en la jerarquía.



5.5. Transform y física

Aunque se puede mover un objeto modificando transform.position, cuando interviene un **Rigidbody2D**, lo correcto es usar:

- rb.MovePosition(...)
- rb.velocity = ...

Para evitar saltarse el motor físico.

Aun así, en scripts como **Bala**, se usa Transform porque el movimiento es simple y no necesita físicas:

```
transform.position += velocidad * transform.right *
Time.deltaTime;
```

5.6. Transform como referencia en otros scripts

Es habitual que scripts guarden o utilicen Transforms de otros objetos.

Ejemplo del script **Enemigo**, usando su propia posición para instanciar proyectiles:

```
Instantiate(prefabBalaE, transform.position + Vector3.down,
Quaternion.Euler(0, 0, -90));
```

Ejemplo del script **Humano**, donde se usa el Transform como origen de un raycast:

```
RaycastHit2D hit = Physics2D.Raycast(transform.position,
Vector2.down, distanciaSensorSuelo, mascaraSuelo);
```

5.7. Métodos útiles relacionados con Transform

LookAt

Hace que un objeto apunte a otro:

```
transform.LookAt(objetivo);
```

TransformDirection / InverseTransformDirection

Convierte direcciones entre espacios local y global.

TransformPoint / InverseTransformPoint

Convierte posiciones entre espacios.

5.8. Resumen

- **Posición**: dónde está el objeto
- **Rotación**: hacia dónde apunta
- **Escala**: qué tamaño tiene
- **Padre–hijo**: cómo se relaciona con otros objetos
- **Vector3**: base para todos los cálculos
- **Transform**: núcleo de cualquier comportamiento dinámico